

Soluciones - TP N° 4

Administración y Seguridad

Parte 1: El Perfil Gerencial (RBAC y Vistas de Reporte)

Resolución SQL esperada:

-- 1 y 2. Roles y Usuarios

```
CREATE ROLE rol_gerente;
```

```
CREATE USER gerente_ana WITH PASSWORD 'Directorio2026';
```

```
GRANT rol_gerente TO gerente_ana;
```

-- 3. Creación de Vistas

-- Vista 1

```
CREATE VIEW v_reporte_ingresos_sucursal AS
```

```
SELECT s.nombre, SUM(p.importe) as total_ingresos
```

```
FROM pedidos p
```

```
JOIN sucursales s ON p.id_sucursal = s.id_sucursal
```

```
GROUP BY s.nombre;
```

-- Vista 2

```
CREATE VIEW v_capital_invertido_categoria AS
```

```
SELECT categoria, SUM(stock * precio_unitario) as capital_invertido
```

```
FROM articulos
```

```
GROUP BY categoria;
```

-- Vista 3

```
CREATE VIEW v_metricas_pedidos AS
```

```
SELECT estado, COUNT(*) as cantidad_pedidos
```

```
FROM pedidos
```

```
GROUP BY estado;
```

-- 4. Permisos

```
GRANT SELECT ON v_reporte_ingresos_sucursal TO rol_gerente;
```

```
GRANT SELECT ON v_capital_invertido_categoria TO rol_gerente;
```

```
GRANT SELECT ON v_metricas_pedidos TO rol_gerente;
```

-- 5. Prueba

```
SET ROLE gerente_ana;
```

```
SELECT * FROM v_reporte_ingresos_sucursal; -- Funciona
```

```
SELECT * FROM pedidos; -- Da error: permission denied
```

```
RESET ROLE;
```

Respuesta a Pregunta 1:

Combinar Vistas con RBAC es una herramienta de Seguridad porque permite aplicar el Principio de Privilegio Mínimo limitando el acceso a nivel de filas y columnas. El usuario no

recibe acceso a los datos crudos (tablas base), sino a un subconjunto procesado. En este ejercicio esto garantiza que el gerente solo vea agregaciones (sumas, conteos) y no pueda vulnerar la privacidad accediendo a qué compró exactamente el cliente X.

Parte 2: Criptografía para Datos Sensibles

Resolución SQL esperada:

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;
```

-- 1. Alterar tabla

```
ALTER TABLE clientes  
ADD COLUMN telefono_emergencia_cifrado BYTEA,  
ADD COLUMN pin_entrega_hash TEXT;
```

-- 2. Guardar datos cifrados y hasheados

```
UPDATE clientes  
SET telefono_emergencia_cifrado = pgp_sym_encrypt('555-0199', 'secreto_empresa'),  
    pin_entrega_hash = crypt('4092', gen_salt('bf'))  
WHERE id_cliente = 1;
```

-- 3. Consulta de validación del PIN (Repartidor)

```
SELECT (pin_entrega_hash = crypt('4092', pin_entrega_hash)) AS pin_correcto  
FROM clientes  
WHERE id_cliente = 1;
```

-- 4. Consulta para recuperar el teléfono

```
SELECT pgp_sym_decrypt(telefono_emergencia_cifrado, 'secreto_empresa') AS  
telefono_plano  
FROM clientes  
WHERE id_cliente = 1;
```

Respuesta a Pregunta 2:

a) No podrá llamar. El teléfono está cifrado con un algoritmo simétrico. Aunque tenga el archivo de la base de datos, necesita la clave 'secreto_empresa' (que usualmente está en el backend de la aplicación, no en la BD) para descifrar el BYTEA.

b) Tampoco podrá saber los PINs. El hash es matemáticamente irreversible (unidireccional). A diferencia del teléfono, ni siquiera teniendo la BD entera el empleado puede hacer "ingeniería inversa" para saber que el hash corresponde a '4092'.

Parte 3: Seguridad Operativa - Limitación de Updates

Resolución SQL esperada:

-- 1. Revocar (limpiar estado anterior)

REVOKE ALL ON pedidos FROM rol_vendedor;

-- 2. Lectura e Inserción general

GRANT SELECT, INSERT ON pedidos TO rol_vendedor;

-- 3. Update granular

GRANT UPDATE (estado) ON pedidos TO rol_vendedor;

Respuesta a Pregunta 3:

El CHECK (importe >= 0) es una regla de Integridad de Dominio / Lógica de Negocio.

Protege a la base de datos de datos matemáticamente inválidos (ej. un importe negativo), sin importar quién intente hacerlo (incluso el DBA).

El GRANT UPDATE (estado) es una regla de Seguridad de Acceso (Autorización). Protege la base de datos de modificaciones no autorizadas basadas en quién es el usuario. Evita que un rol manipule columnas sensibles (como el importe) para las cuales no tiene competencia laboral, aplicando el Principio de Privilegio Mínimo.

Parte 4: Ciclo de vida y Baja Segura

Resolución SQL esperada:

-- No hacemos DROP ROLE rol_gerente porque se reutilizará

REVOKE rol_gerente FROM gerente_ana;

DROP OWNED BY gerente_ana;

DROP USER gerente_ana;

Respuesta a Pregunta 4:

El SGBD prohíbe el DROP USER directo para mantener la Integridad del Catálogo del sistema. Si el motor simplemente lo borrara ignorando sus relaciones, ocurrirían dos cosas:

1) Los objetos creados por ese usuario quedarían "huérfanos" (sin dueño legal), imposibilitando su futura gestión. 2) La Matriz de Acceso (DAC) quedaría corrupta y con inconsistencias, manteniendo en sus registros internos permisos asignados a un ID de usuario que ya no existe (lo cual además es un riesgo de seguridad si el ID se recicla).